MODULE 18 · DOMAIN 2 - SECURITY

AWS KMS and Encryption in Practice

Envelope encryption, key policies, grants – and why a key policy can deny an administrator.

DOMAIN 2 of 4 EXAM WEIGHT 26% MODULE 18 of 34

admin@sthithapragna.com

KMS never sees your data. It protects the **key** that protects your data, and that indirection is the whole design.

The three kinds of KMS key

TYPE	WHO CONTROLS IT
Customer managed	Yours. You set the key policy, rotation, aliases and deletion
AWS managed	Created by a service, named aws/s3 or similar. You cannot edit its policy
AWS owned	Owned by AWS and shared across accounts. You never see it at all

ON THE EXAM

Only customer managed keys give you audit control, cross-account sharing and configurable rotation. Any scenario needing those rules out the other two.

What a KMS key costs

- A customer managed key costs about **1 dollar a month**, plus a charge per request.
- Rotation adds up to **2 dollars a month**, capped after the second rotation.
- AWS managed keys are **free** to keep. You still pay for the requests.
- A key per tenant sounds tidy and becomes expensive fast at thousands of tenants.
- **Encryption context** often isolates tenants adequately without a key each.

> ON THE EXAM
>
> The per-request charge is what makes high-volume workloads expensive, not the monthly key fee. That is why Bucket Keys and caching matter.

Symmetric and asymmetric keys

- **Symmetric** is the default: one key encrypts and decrypts, and it never leaves KMS.
- **Asymmetric** gives a public and private pair. The public key can be downloaded and used outside AWS.
- Asymmetric supports encryption, or **signing and verification**, but a key does one or the other.
- **HMAC** keys generate and verify message authentication codes.
- **Automatic rotation works only on symmetric keys.** Asymmetric and HMAC keys cannot rotate automatically.

> ON THE EXAM
>
> Choose asymmetric only when the other party cannot call KMS. If both sides can, symmetric is simpler, cheaper and rotatable.

The 4 KB limit

- **Encrypt** and **Decrypt** handle at most **4 KB** of data directly.
- That is fine for a password, a token or another key. It is useless for a file.
- Every call is a network round trip, so encrypting a large file directly would be impossibly slow anyway.
- So KMS encrypts a **data key**, and the data key encrypts your data locally.
- **ReEncrypt** moves ciphertext from one KMS key to another without exposing plaintext.

> ON THE EXAM
>
> 4 KB is the number that forces the design. Any scenario about encrypting a file with KMS is really asking whether you know about envelope encryption.

Envelope encryption

01

Generate

Call GenerateDataKey. KMS returns a plaintext key and an encrypted copy.

02

Encrypt

Encrypt your data locally with the plaintext key.

03

Discard

Wipe the plaintext key from memory. Store the encrypted copy beside the data.

04

Decrypt later

Send the encrypted key to KMS, get the plaintext back, decrypt the data.

ON THE EXAM

Step three is the point. The plaintext key exists only in memory, briefly; what is stored is useless without a call to KMS.

The data key operations

- **GenerateDataKey** returns both plaintext and ciphertext. The normal case.
- **GenerateDataKeyWithoutPlaintext** returns only the ciphertext.
- Use it when a component prepares keys for later use and must never see plaintext itself.
- **Decrypt** takes the encrypted data key and returns the plaintext key.
- The ciphertext blob records which KMS key made it, so Decrypt needs no key ID for symmetric keys.

> ON THE EXAM
>
> That last point matters: any principal with kms:Decrypt on the right key can decrypt, without being told which key it is. The blob says so.

Key policies

- **A key policy must explicitly allow access.** An identity policy alone is never enough.
- That is true even within the same account, unlike every other resource policy.
- So a key with an empty policy is unusable by everyone except the root user.
- Every key has exactly one key policy. It cannot be deleted, only edited.
- Locking yourself out is possible, and recovering requires AWS Support.

> READ THIS TWICE
>
> This is why KMS troubleshooting is its own skill. Granting kms:Decrypt in an IAM policy achieves nothing if the key policy does not also permit it.

The default key policy

- The default policy gives the **account root** full access to the key.
- That single statement enables IAM policies in the account to grant key access.
- Remove it and IAM policies stop working entirely for that key.
- Console key creation also adds explicit **key administrator** and **key user** statements.
- Key administrators manage the key. **Key users use it.** Being an administrator does not let you decrypt.

> ON THE EXAM
>
> The last line surprises people. A key administrator can rotate, tag and delete a key without ever being able to decrypt anything with it. That separation is deliberate.

Grants

- A **grant** gives a principal specific key operations without editing the key policy.
- Created with **CreateGrant**, removed with **RetireGrant** or **RevokeGrant**.
- Scoped to named operations, and can carry **encryption context constraints**.
- AWS services use grants constantly. An encrypted EBS volume creates one behind the scenes.
- Grants are **eventually consistent**. Use the returned grant token for immediate use.

ON THE EXAM

Grants suit temporary or programmatic access; key policies suit stable, long-term permissions. Adding a grant does not touch the key policy at all.

Key policy or grant?

KEY POLICY

Stable permissions

- One document per key
- Long-lived, reviewed access
- Changes require editing the policy
- Visible in one place
- The primary access control

GRANT

Temporary or programmatic

- Many grants per key
- Created and retired at runtime
- No policy edit required
- Scoped to specific operations
- How AWS services delegate to themselves

THE TIEBREAKER

'Temporary access that can be revoked without changing the key policy' is a grant, every time. Anything permanent belongs in the policy.

Encryption context

- Key-value pairs supplied at encrypt time and required, identically, at decrypt time.
- It is **not encrypted**. It is authenticated, so it cannot be changed without breaking decryption.
- It appears in **CloudTrail**, which makes decrypt calls far easier to audit.
- It can be enforced in a key policy or grant using the **kms:EncryptionContext** condition key.
- Typical use: bind ciphertext to a tenant or a document, so it cannot be decrypted in another context.

> ON THE EXAM
>
> Because it is logged rather than encrypted, never put anything sensitive in an encryption context. Put an identifier there, not a secret.

Condition keys for KMS

- **kms:ViaService** allows use only through a named service, such as s3.eu-west-1.amazonaws.com.
- So a principal can decrypt objects through S3 but never call KMS Decrypt directly.
- **kms:EncryptionContext:key** requires a specific context value.
- **kms:CallerAccount** and **kms:GrantIsForAWSResource** constrain who and what.
- These make a key policy far tighter than a bare list of principals.

> ON THE EXAM
>
> ViaService is the most useful of these: it turns 'may use this key' into 'may use this key only for this service', which is usually what you actually meant.

Aliases

- A friendly name, always prefixed **alias/**, pointing at one key.
- Applications reference the alias, so the underlying key can change without a code change.
- That is what makes **manual rotation** workable: create a new key, repoint the alias.
- Aliases are **Regional**. The same alias in another Region is a different key.
- AWS managed keys use reserved aliases such as **alias/aws/s3**.

> ON THE EXAM
>
> Referencing aliases rather than key IDs is what makes key changes possible at all. Hard-coded key ARNs make rotation a code change.

Automatic key rotation

- KMS generates new **backing key material**. The key ID, ARN and policy stay the same.
- **Old key material is retained**, so existing ciphertext still decrypts. Nothing is re-encrypted.
- New encryption uses the newest material automatically. Applications notice nothing.
- Symmetric customer managed keys only. Not asymmetric, not HMAC, not custom key stores.
- **AWS managed keys rotate annually** and the schedule cannot be changed.

> ON THE EXAM
>
> The critical point: rotation does not re-encrypt anything. Old data keeps decrypting under retained material, which is why enabling rotation is safe.

Currency note: rotation became configurable

- **The examinable answer:** automatic rotation happens **once a year**.
- Since **12 April 2024** the period is configurable from **90 days to 2,560 days**, about seven years.
- **RotateKeyOnDemand** triggers an immediate rotation, up to **10 times** per key.
- Rotation history is now visible, which matters for compliance evidence.
- Pricing changed too: rotation charges are capped after the second rotation.

> ON THE EXAM
>
> 365 days is still the default and still the exam answer. The change is that it is a default rather than the only option.

Deleting a key

- You **schedule** deletion with a waiting period of **7 to 30 days**. Default 30.
- During the wait the key is unusable, so failures surface before anything is destroyed.
- Deletion can be cancelled at any point during the waiting period.
- Once deleted, **every ciphertext under that key is permanently unrecoverable**.
- Prefer **disabling** the key. It has the same effect and is instantly reversible.

> ON THE EXAM
>
> Alarm on the ScheduledKeyDeletion CloudTrail event. A key scheduled for deletion is one of the few genuinely irreversible actions in AWS.

Multi-Region keys

- A primary key plus **replicas** in other Regions, sharing the same key material and key ID.
- Ciphertext encrypted in one Region **decrypts in another**, with no re-encryption.
- **Ordinary KMS keys are Regional.** Two separate keys cannot decrypt each other's ciphertext.
- Each replica has its own key policy, grants and tags, managed independently.
- This is not automatic. A key must be created as multi-Region from the start.

> ON THE EXAM
>
> The classic failure: cross-Region replication or disaster recovery fails because the target Region's key is a different key. Multi-Region keys are the specific fix.

Server-side encryption options

OPTION	WHAT IT MEANS FOR KMS
SSE-S3	An AWS owned key. No KMS calls, no KMS cost, no audit trail
SSE-KMS	A KMS key. CloudTrail records every use, and access can be revoked
DSSE-KMS	Two independent layers of KMS encryption. Doubles the KMS calls
SSE-C	You send the key with every request. KMS is not involved
Client-side	You encrypt before upload, typically with the AWS Encryption SDK

ON THE EXAM

The reason to choose SSE-KMS over SSE-S3 is control and audit, not stronger encryption. Both use AES-256.

S3 Bucket Keys

- By default, SSE-KMS makes a KMS call for **every object** written and read.
- At scale that is expensive, and it consumes the account's shared KMS request quota.
- An **S3 Bucket Key** is a short-lived bucket-level key derived once from your KMS key.
- S3 then uses it for many objects, cutting KMS requests by up to **99%**.
- Enabled per bucket, and it works with existing keys with no application change.

> ON THE EXAM
>
> Any scenario about KMS throttling or cost on a high-volume S3 workload is pointing at Bucket Keys. It is a configuration change, not a redesign.

Quotas and throttling

- KMS request quotas are **shared across the account and Region**, not per key.
- So one heavy workload can throttle unrelated services that also use KMS.
- Exceeding it returns **ThrottlingException**, which the SDKs retry with backoff.
- Reduce calls with **S3 Bucket Keys**, with **data key caching**, or by requesting a quota increase.
- Cryptographic operations are the metered ones. Describing a key is a separate, higher quota.

> ON THE EXAM
>
> The shared quota is the surprising part. A batch job encrypting millions of objects can cause throttling in a completely unrelated application.

Data key caching

- The **AWS Encryption SDK** can cache data keys and reuse them across many encryptions.
- That cuts KMS calls dramatically for high-throughput client-side encryption.
- It **weakens isolation**: more data ends up protected by the same data key.
- Limits bound the risk: maximum age, maximum messages and maximum bytes per cached key.
- AWS is explicit that this trades some security for performance. Set the limits deliberately.

> ON THE EXAM
>
> An honest trade-off worth knowing. Caching is correct for high-volume workloads with modest sensitivity, and wrong when each item must be independently protected.

Where key material comes from

- **KMS generated** is the default, and the only one supporting automatic rotation.
- **Imported**, sometimes called bring your own key, when you must control key generation.
- Imported material can **expire**, and if you lose your copy the key is unrecoverable.
- **CloudHSM custom key store** keeps material in a HSM cluster you control.
- **External key store**, or XKS, keeps material in a key manager outside AWS entirely.

> ON THE EXAM
>
> Each step away from KMS-generated adds control and adds operational burden. XKS means AWS cannot decrypt your data even in principle - and neither can you if your key manager is down.

KMS or CloudHSM?

AWS KMS

Managed and integrated

- Multi-tenant, FIPS 140-3 Level 3 validated
- Integrated with most AWS services
- AWS manages availability and patching
- Priced per key and per request
- The default for almost everything

AWS CLOUDHSM

Single-tenant hardware

- Dedicated HSM cluster you control
- You manage users, keys and backups
- Supports PKCS#11 and JCE directly
- Priced per HSM hour, considerably more
- For strict compliance or non-AWS workloads

THE TIEBREAKER

Choose CloudHSM only when a requirement names single-tenant hardware, or when an application needs PKCS#11. Otherwise KMS, every time.

Certificates and encryption in transit

- **ACM** issues and renews public TLS certificates **free**, and renews them automatically.
- ACM public certificates **cannot be exported**. The private key never leaves AWS.
- They attach to CloudFront, ALB, API Gateway and App Runner, not to an EC2 instance.
- For CloudFront or an edge-optimised API, the certificate must be in **us-east-1**.
- **ACM Private CA** issues internal certificates, and those can be exported.

> ON THE EXAM
>
> The non-exportable rule is the common trap: you cannot install an ACM public certificate on an EC2 instance or an on-premises server.

Diagnosing a KMS AccessDenied

- Does the **key policy** allow the principal? Nothing works without this.
- Does the **identity policy** allow the kms action? Both are required, unlike other services.
- Is a **grant** needed, or was one retired?
- Does the **encryption context** match what was used at encrypt time?
- Is the key in **another Region**, or scheduled for deletion, or simply disabled?

> ON THE EXAM
>
> CloudTrail records every KMS call with the encryption context and the principal, which usually identifies the gap faster than reading policies.

Monitoring key use

- Every KMS call appears in **CloudTrail**, including the encryption context and the caller.
- That is how you prove which principal decrypted what, and when.
- Alarm on **ScheduledKeyDeletion**. It is the one action you cannot undo after the window.
- Watch for **ThrottlingException** volume, which signals the shared quota is under pressure.
- **DisableKey** and key policy changes are also worth alarming on.

> ON THE EXAM
>
> CloudTrail auditability is the whole reason to choose SSE-KMS over SSE-S3. If nobody watches the trail, that reason evaporates.

Two of these turn on the same fact: **a key policy must allow it, whatever the IAM policy says**.

> A Lambda function must encrypt 40 MB report files before writing them to Amazon S3. The team wants to use a customer managed KMS key so that key use is auditable and access can be revoked centrally.
>
> A first attempt called the KMS Encrypt API with the file contents and failed.

How should the encryption be implemented?

A Split the file into 4 KB chunks and call Encrypt on each one

B Request a quota increase for the KMS Encrypt payload size

C Use an asymmetric KMS key, which has no payload limit

D Call GenerateDataKey, encrypt the file locally with the plaintext key, and store the encrypted data key with the object

◆ PAUSE AND DECIDE

How should the encryption be implemented?

A 4 KB chunks

40 MB is about 10,000 chunks, so 10,000 KMS calls per file. Slow, expensive, and it would throttle the account.

B Quota increase

4 KB is a hard API limit, not an adjustable quota. No request changes it.

C Asymmetric key

Asymmetric keys have a smaller payload limit, not a larger one. They are also far slower for bulk data.

D Envelope encryption

The intended pattern. One KMS call per file, encryption happens locally at full speed, and the stored encrypted data key is useless without KMS.

THE SIGNAL

Every wrong answer tries to make KMS encrypt the data. The whole design is that it never does - it encrypts the key that encrypts the data.

> Account A must read objects from a bucket in Account B that are encrypted with SSE-KMS using a customer managed key in Account B. The bucket policy in Account B allows the role in Account A, and the role's identity policy in Account A allows both s3:GetObject and kms:Decrypt on the key ARN.
>
> Downloads still fail with AccessDenied from KMS, not from S3.

What is missing?

A. The KMS key policy in Account B must explicitly allow the role in Account A to use the key

B. The key must be converted to a multi-Region key

C. Account A must create its own KMS key with the same key material

D. The bucket must be switched from SSE-KMS to SSE-S3

◆ PAUSE AND DECIDE

What is missing?

A Key policy must allow the role

KMS requires an explicit allow in the key policy. The identity policy grants the intent; the key policy grants the access. Both are needed.

B Multi-Region key

Multi-Region solves cross-Region, not cross-account. Nothing here suggests two Regions are involved.

C Duplicate the key material

Key material cannot be copied out of KMS, and a separate key could never decrypt this ciphertext anyway.

D Switch to SSE-S3

It would work by abandoning the audit and revocation control the team chose SSE-KMS for. Solving the problem by removing the requirement.

THE SIGNAL

The error coming from KMS rather than S3 is the clue. S3 access is already correct; the key is refusing separately, which is the behaviour unique to KMS.

> A data pipeline writes about 8 million objects a day to a bucket using SSE-KMS with a customer managed key. Teams across the account report intermittent ThrottlingException errors from unrelated services that also use KMS, and the KMS bill has grown sharply.
>
> The encryption requirement, including CloudTrail auditability of key use, cannot be relaxed.

What should the team do?

A. Switch the bucket to SSE-S3 to remove KMS from the path

B. Create a separate KMS key for the pipeline so it has its own request quota

C. Enable S3 Bucket Keys on the bucket

D. Add exponential backoff to the pipeline and accept the throttling

◆ PAUSE AND DECIDE

What should the team do?

A Switch to SSE-S3

Removes KMS calls and removes the CloudTrail audit trail with them, which the scenario says cannot be relaxed.

B A separate key

The tempting one, and it rests on a misunderstanding. KMS request quotas are per account and Region, not per key, so a second key shares the same quota.

C S3 Bucket Keys

Cuts KMS requests by up to 99% by deriving a bucket-level key, keeping SSE-KMS and its audit trail. A per-bucket setting with no code change.

D Backoff

Manages the symptom for the pipeline while unrelated services keep being throttled. It also does nothing about the bill.

THE SIGNAL

Option B is the instructive wrong answer: quotas are per account and Region. Splitting keys is a natural instinct and simply does not help.

A third-party analytics vendor needs to decrypt a specific set of exported files for a two-week engagement. The files are encrypted with a customer managed key whose policy is reviewed quarterly and change-controlled.

Security wants the access scoped to decryption only, revocable at any moment, and does not want the key policy modified for a temporary arrangement.

How should the access be granted?

A Add a statement to the key policy allowing the vendor, and remove it after two weeks

B Create a KMS grant for the vendor's role, scoped to Decrypt, and retire it when the engagement ends

C Share the plaintext data key with the vendor directly

D Create a second KMS key for the vendor and re-encrypt the files under it

◆ PAUSE AND DECIDE

How should the access be granted?

A Edit the key policy

Directly contradicts the requirement. It also puts a temporary change through a quarterly change-control process twice.

B A KMS grant scoped to Decrypt

Grants exist for exactly this: temporary, operation-scoped access created and retired at runtime, with no key policy change.

C Share the plaintext data key

Hands over a key that cannot be revoked, cannot be audited, and protects every file it ever encrypted.

D Second key and re-encrypt

Works, at the cost of re-encrypting the whole dataset and maintaining a second key. ReEncrypt exists, but this is disproportionate.

THE SIGNAL

Three requirements - temporary, operation-scoped, no policy change - describe a grant precisely. That combination appears whenever grants are the answer.

What to carry forward

- **KMS encrypts keys, not data.** Over 4 KB means envelope encryption with GenerateDataKey.
- **A key policy must explicitly allow access**, even same-account. An identity policy alone never suffices.
- **Grants for temporary access**, key policies for permanent access.
- Rotation changes backing material only. Nothing is re-encrypted, and old ciphertext still decrypts.
- KMS quotas are **per account and Region**. Bucket Keys and data key caching reduce calls.

> ON THE EXAM
>
> Next: Secrets Manager, Parameter Store and application configuration - where the things KMS protects actually live.